Attention-Preserving Post-Training Quantization for Transformer Models

Pooya Emami Varzeghani (404300409)
Mohammad Mosayebzadeh (402110854)
Department of Electrical Engineering, Sharif University of Technology

August 2026

Abstract

Post-training quantization (PTQ) reduces the memory footprint of large language models by lowering weight precision. Standard methods like GPTQ and AWQ treat each weight matrix independently, ignoring the coupled nature of attention. We implement an attention-aware framework that jointly calibrates Query, Key, Value, and Output projection matrices using a combined output-reconstruction and temperature-annealed KL-divergence loss, optimized via a straight-through estimator. We reuse the same loss as a sensitivity metric for mixed-precision allocation, solving the bit-allocation problem via dynamic programming over a five-point sensitivity curve per block. On GPT-2 and Llama-3.2-1B with Open-Platypus and WikiText-2, our method achieves the best 4-bit and 3-bit perplexity on GPT-2/Open-Platypus, remains competitive on Llama-3.2-1B, and our mixed-precision allocation recovers near-FP32 perplexity at a 5-bit average budget, while using 4-10$\times$ less GPU memory than baselines.

Code: `github.com/pooya-emami/deep-final-project`

1 Introduction

Transformer models have become the dominant architecture in modern deep learning. Since the original Transformer [13], this architecture has been scaled to create large language models with billions of parameters. Models like GPT, LLaMA, and OPT have demonstrated strong performance across various natural language tasks, but their large size makes deployment difficult on devices with limited memory.

Quantization has been shown effective for reducing model size [1, 3]. By reducing weight precision from 16-bit floats to 4-bit integers, memory footprint can be reduced by roughly 75%. Post-Training Quantization (PTQ) is particularly attractive because it does not require retraining, using only a small calibration dataset to determine quantization parameters.

Existing PTQ methods like GPTQ [1] and AWQ [3] minimize reconstruction error for each weight matrix independently:

$$\min_{\hat{\mathbf{W}}} \|\mathbf{X}\mathbf{W} - \mathbf{X}\hat{\mathbf{W}}\|_F^2 \tag{1}$$

where $\mathbf{W}$ is the original weight matrix, $\hat{\mathbf{W}}$ is the quantized version, and $\mathbf{X}$ is the layer input.

This formulation ignores the coupled nature of attention. The Query ($\mathbf{W}_Q$), Key ($\mathbf{W}_K$), Value ($\mathbf{W}_V$), and Output ($\mathbf{W}_O$) projections interact through softmax:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{2}$$

Here $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, and $\mathbf{V} = \mathbf{X}\mathbf{W}_V$. Quantizing each matrix independently ignores these interactions, and small errors in $\mathbf{W}_Q, \mathbf{W}_K$ can compound in the attention output.

We implement a method that accounts for this coupling. We include $\mathbf{W}_O$ in the joint objective and validate on GPT-2 and Llama-3.2-1B. The joint loss is reused as a sensitivity metric for mixed-precision allocation via dynamic programming over a full five-point sensitivity curve.

Our main contributions are:

- A joint calibration objective over $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$ combining output-reconstruction and temperature-annealed KL-divergence loss, optimized via STE/Adam.
- Using the same objective as a per-block sensitivity metric with exact DP for MCKP bit-allocation over a 5-point curve.
- A forward-pass implementation across GPT-2 and Llama attention mechanisms.
- Analysis of diminishing returns across calibration passes and memory/accuracy trade-offs from 3.5- to 5-bit average budgets.

2 Related Work

Quantization methods generally fall into Quantization-Aware Training (QAT) and Post-Training Quantization (PTQ). QAT methods [12] train with quantization in the loop, which preserves accuracy but is often impractical for LLMs due to high computational cost. PTQ methods quantize a pre-trained model in a single pass using a small calibration set and have become the standard approach for LLM quantization.

Early PTQ methods like AdaRound [10] introduced data-dependent rounding by annealing a penalty term that encourages weights to move toward quantization grid points. BRECQ [11] incorporated Fisher information into the objective and optimized layers within a residual block jointly. Both methods scaled well only up to about 100M parameters.

GPTQ [1], building on Optimal Brain Quantization (OBQ) [2], was the first to successfully quantize very large language models. GPTQ introduces a fixed quantization order and lazy batch updates to make the algorithm efficient. After quantizing weight w_q, the remaining weights are updated as:

$$\delta_F = -\frac{w_q - \text{quant}(w_q)}{[\mathbf{H}_F^{-1}]_{qq}} \cdot (\mathbf{H}_F^{-1})_{:q} \tag{3}$$

where $\mathbf{H}_F = 2\mathbf{X}_F\mathbf{X}_F^T$ is the layer Hessian. This second-order approach allows GPTQ to achieve high accuracy at 3–4 bits.

AWQ [3] takes a different approach, observing that only a small fraction of weights (0.1–1%) are salient and dominate performance. These weights are identified via activation magnitude rather than weight magnitude. AWQ then protects them with per-channel scaling:

$$Q(w \cdot s) \cdot \frac{x}{s} \approx Q(w) \cdot x. \tag{4}$$

This scaling reduces quantization error without requiring mixed-precision hardware. AWQ is training-free and generalizes well across domains.

SmoothQuant [4] takes a closely related view, migrating quantization difficulty from activations to weights via similar per-channel scaling, enabling joint weight-activation quantization. Outlier Suppression+ [5] further extends this with channel-wise shifting.

APTQ [6] is closest to our work: it is the first method to explicitly account for the attention mechanism in LLM quantization. APTQ uses attention-based gradients with a Hessian-based sensitivity metric, formulating the objective as:

$$\arg\min_{\hat{\mathbf{W}}} \|F(\mathbf{W}) - F(\hat{\mathbf{W}})\|_2^2, \tag{5}$$

where F denotes the attention output. APTQ approximates the Hessian with a Gauss-Newton (Jacobian outer-product) form and uses its trace for mixed-precision allocation.

OmniQuant [7] introduces learnable weight clipping and equivalent transformations for both weight-only and weight-activation quantization. MixFrag [8] proposes a fragility-guided mixed-precision framework for Vision Transformers, estimating component-level fragility via KL divergence between full-precision and isolated-quantized outputs and solving an MCKP for bit allocation.

Our approach differs from APTQ by using direct loss minimization rather than a Gauss-Newton-style Hessian approximation, and our joint loss combines output reconstruction with temperature-annealed KL divergence rather than a single squared-error term. Relative to MixFrag, we extend the KL-based MCKP sensitivity idea from vision transformers to autoregressive decoder-only LLMs, use the same loss for calibration, and measure full five-point sensitivity curves rather than interpolating between endpoints. We note, however, that both our sensitivity metric and MixFrag's fragility score measure each block's importance in isolation and then allocate bits via an additive knapsack objective, which does not model interaction between blocks' quantization errors – a limitation we discuss in Sec. 4.6.

3 Methodology

Our implementation follows the AP-Quant approach with LSQ-style [9] scale optimization. The method has two components: joint attention calibration and mixed-precision allocation.

3.1 Joint Attention Calibration

Instead of separate reconstruction losses per matrix, we use a joint objective over $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$:

$$\mathcal{L}_{joint} = \mathcal{L}_{output} + \lambda\, \mathcal{L}_{KL}. \tag{6}$$

The output loss preserves the (post-W_O) context vector:

$$\mathcal{L}_{output} = \frac{\|\mathbf{O} - \hat{\mathbf{O}}\|_F^2}{\|\mathbf{O}\|_F^2 + \epsilon}, \tag{7}$$

normalized for scale-invariance across layers and heads.

The KL loss preserves the attention distribution, but is computed on a temperature-annealed re-normalization of the softmax outputs rather than the raw distributions, i.e.

$$\tilde{\mathbf{P}} = \mathrm{softmax}\left(\frac{\log \mathbf{P}}{\tau}\right), \qquad \tilde{\hat{\mathbf{P}}} = \mathrm{softmax}\left(\frac{\log \hat{\mathbf{P}}}{\tau}\right), \tag{8}$$

$$\mathcal{L}_{KL} = \frac{1}{|\mathcal{Q}|} \sum_{i \in \mathcal{Q}} D_{KL}\left(\tilde{\mathbf{P}}_i \,\|\, \tilde{\hat{\mathbf{P}}}_i\right), \tag{9}$$

where $\mathbf{P}, \hat{\mathbf{P}}$ are the original and quantized attention distributions, $\mathcal{Q}$ is the set of causally-valid query positions, and τ anneals from 2.0 to 1.0 over calibration. Annealing τ from a softer to a sharper distribution lets early optimization steps match coarse attention structure before refining fine-grained peaks, which we found more stable than matching the raw (already very peaked) softmax outputs from the first step.

Weights are quantized via a straight-through estimator (STE):

$$\hat{\mathbf{W}} = \Delta \cdot \mathrm{clip}\left(\left\lfloor \frac{\mathbf{W}}{\Delta} \right\rceil, -Q_{\max}, Q_{\max}\right), \qquad Q_{\max} = 2^{b-1} - 1, \tag{10}$$

where the per-channel scale $\Delta = \mathrm{softplus}(\rho) + \epsilon$ is reparameterized through an unconstrained parameter ρ to guarantee positivity without hard clamping. Scales are optimized via Adam ($\mathrm{lr} = 10^{-3}$, cosine-annealed, gradient norm clipped to 10) for 200 steps per layer.

The KL weight λ is not fixed, but re-estimated online every 10 steps from the running ratio of the two loss terms:

$$\lambda_{t+1} = \mathrm{clip}\left(0.5\,\lambda_t + 0.5\,\frac{\mathcal{L}_{output}}{\mathcal{L}_{KL} + \epsilon},\ 0.01,\ 10\right), \tag{11}$$

which keeps the two terms on comparable scales without manual tuning per layer.

3.2 Mixed-Precision Allocation

We reuse $\mathcal{L}_{joint}$ as a sensitivity metric. For each attention block i and candidate bit-width $b \in \{2, 3, 4, 8, 16\}$, we quantize block i alone (with FP32 inputs captured upstream under global FP mode, equivalent to isolating block i under full-precision context, following the single-block sensitivity protocol used by APTQ [6] and MixFrag [8]) and measure

$$L_i(b) = \mathcal{L}_{joint}\big|_{\text{block } i \text{ quantized to } b\text{-bit}}. \tag{12}$$

We measure the full five-point curve rather than interpolating between two endpoints, and solve the resulting Multiple-Choice Knapsack Problem exactly via dynamic programming:

$$\min_{\{b_i\}} \sum_{i=1}^{N} L_i(b_i) \quad \text{s.t.} \quad \sum_i C(b_i) \le B, \tag{13}$$

where $C(b_i) = b_i$ is the per-block memory cost and B is the total budget, guaranteeing exactly one bit-width per block under the memory constraint. As discussed in Sec. 4.6, this additive objective assumes blocks contribute independently to total model degradation.

Algorithm 1 APQ: Attention-Preserving Quantization

Require: Model M, calibration data $\mathcal{D}$, target average bit budget B
Ensure: Quantized model M_q with per-block bit-widths $\{b_i^*\}$
Initialize scale parameters Δ_i via RTN for each attention block
Collect FP32 inputs X_i for each block (global FP32 forward pass)
for each block i **do**
  for each candidate $b \in \{2, 3, 4, 8, 16\}$ **do**
    Quantize block i alone to b-bit (RTN-init scale); other blocks unchanged
    Compute $L_i(b) \leftarrow \mathcal{L}_{joint}$ on X_i (Eq. 6)
  end for
end for
Solve MCKP exactly via DP: $\{b_i^*\} \leftarrow \arg\min_{\{b_i\}} \sum_i L_i(b_i)$ s.t. $\sum_i b_i \le B \cdot N$ (Eq. 13)
Re-quantize every block i to b_i^*-bit (fixed for the remainder)
for each pass $p = 1$ to P **do**
  for each block i (in order) **do**
    Collect input X_i from the model with blocks $< i$ already calibrated this pass and blocks $\ge i$ at their current (RTN or previously-calibrated) scale
    $\lambda \leftarrow$ initialize from $\mathcal{L}_{output}/\mathcal{L}_{KL}$ on a sample of X_i; $\tau \leftarrow 2.0$, $t \leftarrow 0$
    while $t < T$ **do**
      Forward block i twice on X_i: full-precision ($\mathbf{P}$, $\mathbf{O}$) and at b_i^*-bit ($\hat{\mathbf{P}}$, $\hat{\mathbf{O}}$)
      Compute $\mathcal{L}_{joint} = \mathcal{L}_{output} + \lambda\, \mathcal{L}_{KL}$ (Eqs. 6–8)
      Backpropagate through the STE; update Δ_i via Adam, clip grad-norm to 10
      if $t \bmod 10 = 0$ **then**
        Update λ per Eq. (11)
      end if
      $\tau \leftarrow \max(1.0, \tau \cdot 0.999)$; $t \leftarrow t + 1$
    end while
    Freeze Δ_i at its best-validation-loss checkpoint
  end for
  Evaluate validation perplexity; keep this pass's scales if best so far
end for
return M_q with scales $\{\Delta_i\}$ and bit-widths $\{b_i^*\}$

3.3 Sequential Calibration

Layers are calibrated sequentially over multiple passes. Bit-widths are fixed once from the MCKP solution; within a pass, blocks are calibrated in order. Each block is calibrated against realistic partially-quantized activations – blocks earlier in the pass are quantized, later blocks sit at their current scales – following GPTQ's sequential error propagation. Temperature starts at $\tau = 2.0$ and decays to 1.0; λ is updated per Eq. (11). This scheme keeps peak GPU memory low relative to full joint calibration.

Algorithm 1 summarizes the full procedure.

4 Experimental Results

4.1 Experimental Setup

We evaluate on GPT-2 (124M) and Llama-3.2-1B (1.2B). Calibration uses 256 samples from Open-Platypus [14] or WikiText-2 [15], tokenized into 128-token chunks and evaluated on a disjoint held-out set. We use HuggingFace Transformers [16] and `llmcompressor` [17] for baselines. Our method uses 200 calibration steps per layer, batch size 8, and 1–3 passes.

Perplexity (PPL) [15] is computed as:

$$\mathrm{PPL} = \exp\left(\frac{\sum_{c=1}^{C}(T_c - 1)\,\mathrm{CE}_c}{\sum_{c=1}^{C}(T_c - 1)}\right), \tag{14}$$

$$\mathrm{CE}_c = -\frac{1}{T_c - 1}\sum_{t=1}^{T_c-1} \log p_\theta\left(x_{t+1}^{(c)} \mid x_{\le t}^{(c)}\right), \tag{15}$$

i.e. the exponentiated token-count-weighted average cross-entropy across C chunks of length T_c.

4.2 Perplexity Comparison at 4 Bits

Table 1 shows 4-bit perplexity. On Open-Platypus, APQ achieves 24.61 for GPT-2 – best among quantization methods (RTN 26.43, AWQ 25.81, GPTQ 24.74), close to FP32 (24.39). For Llama-3.2-1B, GPTQ is best (6.82), followed by APQ (7.02). On WikiText-2, GPTQ generally performs best.

Table 1: Perplexity at 4-bit quantization

Method	Open-Platypus		WikiText-2	
	GPT-2	Llama-3.2-1B	GPT-2	Llama-3.2-1B
FP16/FP32	24.39	6.66	60.68	25.09
RTN	26.43	7.46	69.06	27.14
GPTQ [1]	24.74	**6.82**	**62.13**	**26.17**
AWQ [3]	25.81	6.93	63.41	27.09
APQ (Ours)	**24.61**	7.02	64.53	26.33

4.3 Effect of Calibration Passes

Table 2 shows sequential calibration passes on GPT-2/Open-Platypus. Each pass improves perplexity: 24.99, 24.79, 24.61. Diminishing returns suggest one or two passes suffice.

Table 2: Effect of calibration passes on GPT-2/Open-Platypus.

Configuration	Perplexity
FP32 Baseline	24.39
RTN (4-bit)	26.43
APQ – Pass 1	24.99
APQ – Pass 2	24.79
APQ – Pass 3	24.61

4.4 Bit-Width Comparison

Table 3 compares methods across bit-widths. Higher bit-widths reduce perplexity; gaps are largest at 3-bit. For GPT-2, APQ is best at 3 and 4 bits. For Llama-3.2-1B, APQ trails AWQ at 3-bit, is best at 6-bit, and GPTQ is best at 4-bit.

Table 3: Bit-width comparison on Open-Platypus

Method	3-bit		4-bit		6-bit	
	GPT-2	Llama-3.2-1B	GPT-2	Llama-3.2-1B	GPT-2	Llama-3.2-1B
FP16/FP32	24.39	6.66	24.39	6.66	24.39	6.66
RTN	27.42	7.81	26.43	7.46	25.20	7.06
GPTQ [1]	26.13	7.40	24.74	**6.82**	**24.67**	6.73
AWQ [3]	26.73	**7.21**	25.81	6.93	24.93	6.91
APQ (Ours)	**25.44**	7.37	**24.61**	7.02	24.75	**6.71**

4.5 Mixed-Precision Analysis

Table 4 shows MCKP allocation on GPT-2/Open-Platypus. At 4-bit average, PPL is 24.61; at 3.5-bit it rises to 26.99; at 5-bit it drops to 24.24.

Table 4: Mixed-precision results for GPT-2 on Open-Platypus.

Target Average Bits	Perplexity
FP32 Baseline	23.39
3.5-bit	26.99
4.0-bit	24.61
5.0-bit	24.24

4.6 GPU Memory Usage

Table 5 compares peak GPU memory during calibration on a single T4 GPU (batch size 8, 256 samples). Our method uses less memory than GPTQ and AWQ by optimizing one attention block at a time and not materializing large Hessian matrices.

Table 5: Peak GPU memory usage (GB) for 4-bit calibration on Open-Platypus.

Method	GPT-2	Llama-3.2-1B
GPTQ [1]	1.2	6.2
AWQ [3]	1.9	7.5
APQ (Ours)	0.6	3.1

4.7 Discussion

APQ performs competitively with GPTQ and AWQ, though relative performance depends on model and dataset. On GPT-2 with Open-Platypus, APQ gives the lowest perplexity at 3 and 4 bits. On Llama-3.2-1B, GPTQ generally yields better results, with APQ only outperforming at 6-bit. This suggests the attention-aware objective may be more beneficial for smaller models.

The choice of calibration dataset also matters. APQ performs better on Open-Platypus than WikiText-2, where GPTQ tends to be superior. A possible explanation is that the attention-preserving objective provides greater benefit when token distributions are more varied, as in instruction data.

Several limitations should be acknowledged. The sensitivity metric remains a heuristic; we have not compared it against Fisher information or Hessian trace. The MCKP formulation assumes independent block contributions, ignoring error compounding through the residual stream. We also did not ablate the KL term or W_O inclusion, and report only perplexity, leaving latency and memory for future work.

5 Conclusion

We implemented APQ, a PTQ method that jointly calibrates $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$ using a temperature-annealed output-plus-KL objective, reused for MCKP-based mixed-precision allocation. On GPT-2 with Open-Platypus, APQ gives the best perplexity at 3 and 4 bits, though GPTQ performs better on Llama-3.2-1B and WikiText-2. Mixed-precision allocation approaches FP32 at 5 bits for GPT-2, and calibration passes show diminishing returns after the first pass. Our method also uses 4-10$\times$ less GPU memory than baselines, making it attractive for resource-constrained environments.

Future work includes comparing sensitivity metrics against Fisher information or Hessian trace, testing the additive independence assumption via greedy sequential re-measurement, ablating the KL term and W_O inclusion, measuring latency and memory on INT4/INT8 kernels, and evaluating on larger models.

References

[1] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," *arXiv preprint arXiv:2210.17323*, 2022.

[2] E. Frantar and D. Alistarh, "Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning," *Advances in Neural Information Processing Systems*, 2022.

[3] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration," *Proceedings of Machine Learning and Systems*, vol. 6, 2024.

[4] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," *International Conference on Machine Learning*, 2023.

[5] X. Wei, Y. Zhang, Y. Li, X. Zhang, R. Gong, J. Guo, and X. Liu, "Outlier Suppression+: Accurate Quantization of Large Language Models by Equivalent and Optimal Shifting and Scaling," *arXiv preprint arXiv:2304.09145*, 2023.

[6] Z. Guan, H. Huang, Y. Su, H. Huang, N. Wong, and H. Yu, "APTQ: Attention-aware Post-Training Mixed-Precision Quantization for Large Language Models," *arXiv preprint arXiv:2402.14866*, in *61st ACM/IEEE Design Automation Conference (DAC)*, 2024.

[7] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, "OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models," *International Conference on Learning Representations (ICLR)*, 2024.

[8] M. M. H. Opi, R. I. Ryad, and M. U. Faruk, "MixFrag: Fragility-Guided Mixed-Precision Post-Training Quantization for Vision Transformers," *arXiv preprint arXiv:2607.28589*, 2025.

[9] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned Step Size Quantization," *International Conference on Learning Representations (ICLR)*, 2020.

[10] M. Nagel, R. A. Amjad, M. Van Baalen, C. Louizos, and T. Blankevoort, "Up or Down? Adaptive Rounding for Post-Training Quantization," *International Conference on Machine Learning*, 2020.

[11] Y. Li, R. Gong, X. Tan, Y. Yang, P. Hu, Q. Zhang, F. Yu, W. Wang, and S. Gu, "BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction," *International Conference on Learning Representations*, 2021.

[12] Z. Liu, B. Oguz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, and V. Chandra, "LLM-QAT: Data-Free Quantization Aware Training for Large Language Models," *arXiv preprint arXiv:2305.17888*, 2023.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is All You Need," *Advances in Neural Information Processing Systems*, 2017.

[14] A. Lee, C. Xiao, and W. Chen, "Open-Platypus: An Open-Source Instruction Dataset," 2024.

[15] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer Sentinel Mixture Models," *arXiv preprint arXiv:1609.07843*, 2016.

[16] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, and J. Brew, "HuggingFace's Transformers: State-of-the-art Natural Language Processing," *arXiv preprint arXiv:1910.03771*, 2019.

[17] Neural Magic, "LLM Compressor: Quantization and Pruning for Large Language Models," `https://github.com/vllm-project/llm-compressor`, 2024.